

Facultad de Ciencias UNAM

Estructuras de Datos

Nota3: Tiempos de Ejecución y Funciones de Complejidad

Profesora: Virginia Teodosio Procopio

Ayudantes: Leonardo Gallo Guerrero

José Manuel Madrigal Ramírez

21 de Septiembre de 2026

Análisis de algoritmos

Los tipos de dato abstracto son herramientas teóricas que nos permitirán demostrar la correctez de nuestros programas gracias al contraste entre la definición de las operaciones que hayamos proporcionado y los resultados obtenidos.

Sin embargo, como científicos de la computación no sólo queremos saber si un algoritmo es correcto, sino también si es eficiente. Justamente, la eficiencia es «una medida de qué tan bien son aprovechados los recursos para lograr el objetivo» [1]. En este sentido consideraremos dos recursos: el tiempo que le toma a un algoritmo completar su ejecución y la cantidad de memoria que utiliza. Ahora bien, ¿de qué forma podremos medir cuánto se tarda un algoritmo en realizar una tarea y qué tanta memoria utiliza? Para responder esta pregunta debemos considerar que un algoritmo es:

Una serie de pasos finitos y ordenados, ejecutables en una computadora y que se efectúan sobre un conjunto de datos de entrada para producir una salida determinada.

Es razonable pensar que el tiempo necesario para procesar un conjunto de datos de entrada grande, digamos un arreglo de cien mil entradas, será distinto que procesar un conjunto de entrada pequeño como un arreglo de veinte entradas. Y en general, podríamos considerar otras características sobre los datos de entrada como su estructura, su representación o si se encuentran ordenados. Tales características constituyen la **complejidad de entrada** del algoritmo. Por ahora, para introducir algunas ideas al respecto, consideraremos que la complejidad de entrada se refiere al tamaño de los datos de entrada.

La idea para comparar la eficiencia de un par de algoritmos está en definir una regla de correspondencia que relacione tiempo y complejidad de entrada. Es decir, que dado un algoritmo, sepamos asignar a una determinada complejidad de entrada un recurso utilizado, en particular, que dado el tamaño de una entrada n_0 , obtengamos el tiempo $f(n_0)$ que le toma a nuestro algoritmo poder procesarla. Dicha regla de correspondencia suele recibir el nombre de **función de complejidad** y al obtenerla no sólo podremos tener una mejor perspectiva de la eficiencia de un algoritmo sino que podremos comparar las funciones de complejidad de varios algoritmos destinados a un mismo objetivo y encontrar cuál es el mejor para nuestros propósitos.

Una primera aproximación para deducir la función de complejidad es programar el algoritmo y ejecutarlo con distintas complejidades de entrada. Cada ejecución del algoritmo constituye un experimento y tras graficar nuestros experimentos podríamos considerar qué función matemática se apega más a los datos y establecerla como la función de complejidad de nuestro algoritmo.

Utilizaremos este proceso para encontrar la función de complejidad de distintos algoritmos que resuelven el **problema de suma máxima**: dado un arreglo de números reales, debemos encontrar el subarreglo de suma máxima. Dicho subarreglo no siempre tendrá la mayor longitud pues entre los posibles valores de cada celda pueden haber números negativos que hagan más pequeño el valor total de sumar todos los números contenidos en el subarreglo.

Veamos algunos ejemplos del problema en cuestión:

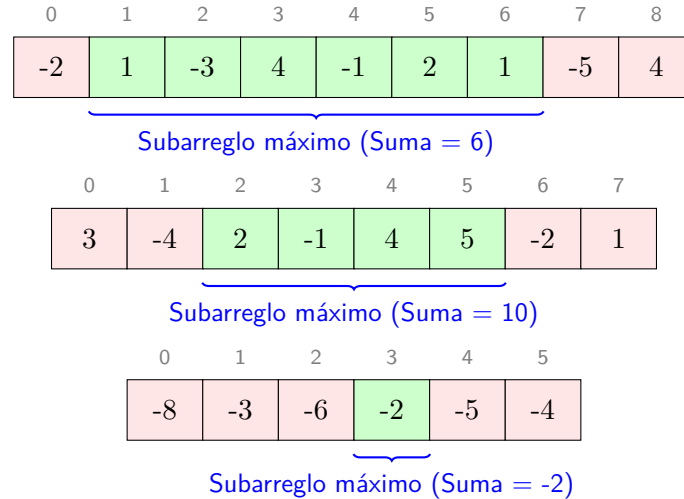


Figura 1: Ejemplos de subarreglos con suma máxima en un arreglo de reales dado.

Ahora sí, exploremos cuatro algoritmos que dan solución a nuestro problema:

MaxSum1 : Este algoritmo consiste en manejar tres índices, los dos primeros delimitan subarreglos y el tercero sirve para sumar los elementos de cada subarreglo. Tras obtener la suma de un nuevo subarreglo, la comparamos con el valor de la que considerábamos ser la suma máxima hasta el momento, y actualizamos este último valor si es necesario:

Algoritmo 1 MAXSUM1(v, n)

```

1:  $sumamax \leftarrow 0$ 
2: for  $i \in \{0, \dots, n-1\}$  do
3:   for  $j \in \{i, \dots, n-1\}$  do
4:      $sumactual \leftarrow 0$ 
5:     for  $k \in \{i, \dots, j\}$  do
6:        $sumactual \leftarrow sumactual + v[k]$ 
7:     end for
8:     if  $sumactual > sumamax$  then
9:        $sumamax \leftarrow sumactual$ 
10:    end if
11:  end for
12: end for
13: return  $sumamax$ 

```

Tras programar el algoritmo y graficar los resultados (cada punto es el resultado de promediar 100 ejecuciones) obtenemos la gráfica de la izquierda, que al linealizarla sacando el cubo de las coordenadas sobre el eje y obtenemos la gráfica de la derecha:

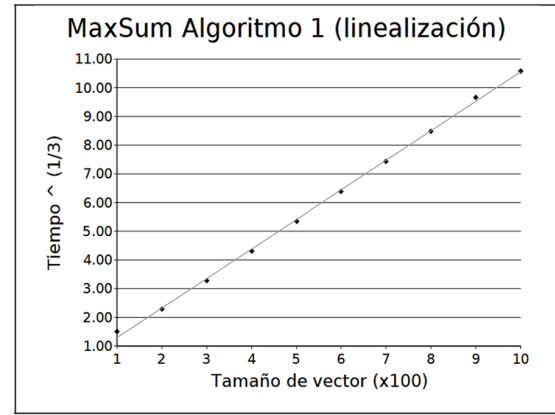
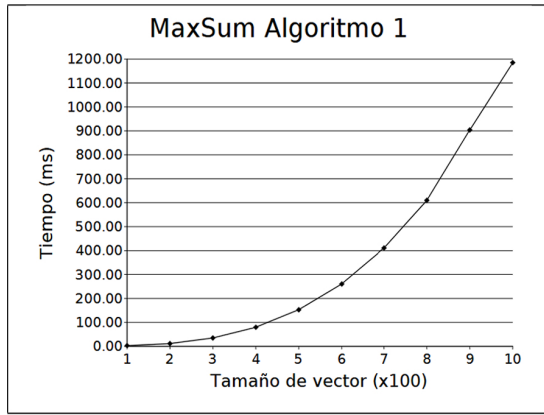


Figura 2: Gráficas de la ejecución de MaxSum1.

MaxSum2: Empezamos por considerar la suma máxima de entre todos los subarreglos que comienzan en la posición 0. Tras guardar ese valor en la variable *sumamax*, obtenemos la suma máxima de entre los subarreglos que comienzan en la posición 1 y lo guardamos en *sumactual*. En seguida comparamos el valor de *sumamax* y *sumactual* y nos quedamos con el mayor guardándolo en la variable *sumamax*. A continuación obtenemos la suma máxima de entre los subarreglos que comienzan en el índice 2 del arreglo y así sucesivamente:

Algoritmo 2 MAXSUM2(v, n)

```

1: sumamax  $\leftarrow$  0
2: for  $i \in \{0, \dots, n-1\}$  do
3:   sumactual  $\leftarrow$  0
4:   for  $j \in \{i, \dots, n-1\}$  do
5:     sumactual  $\leftarrow$  sumactual +  $v[j]$ 
6:     if sumactual > sumamax then
7:       sumamax  $\leftarrow$  sumactual
8:     end if
9:   end for
10: end for
11: return sumamax

```

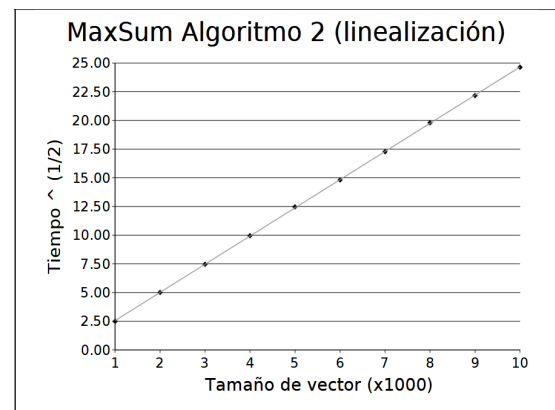
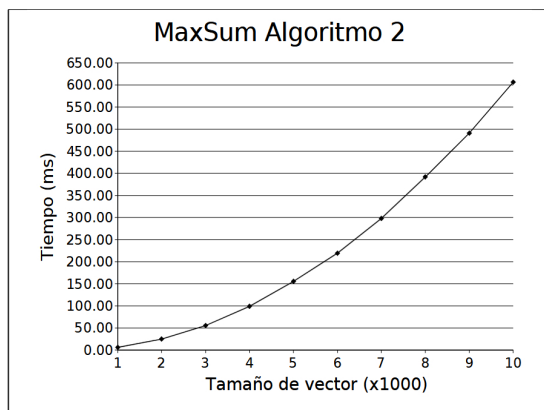


Figura 3: Gráficas de la ejecución de MaxSum2.

MaxSum3: Este algoritmo depende de colocar un índice al inicio y otro índice al final del arreglo y ejecutar el algoritmo recursivo `SUMPARTICION`, que a su vez comienza la búsqueda del subarreglo cuya suma sea máxima dividiendo el arreglo original en mitades. Posteriormente guarda el valor de la suma máxima de ambas mitades combinadas en la variable *sumainterseccion*. Después obtiene la suma máxima de cada mitad por separado guardándolas en las variables *maxpartizq* y *maxpartder* respectivamente. Por último, compara *sumainterseccion*, *maxpartizq* y *maxpartder* para obtener la suma máxima definitiva.

A la izquierda se encuentra la gráfica resultante de implementar y ejecutar el algoritmo varias veces con distintas complejidades de entrada.

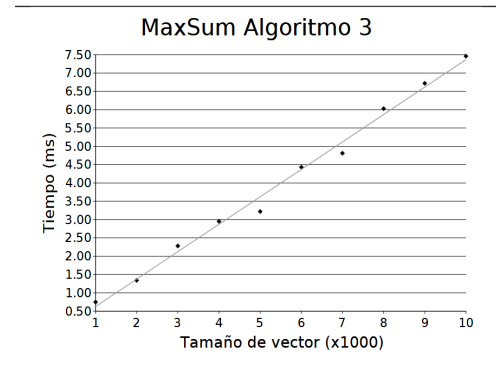


Figura 4: Gráfica de la ejecución de MaxSum3 (cada punto es el promedio de 100 ejecuciones).

Algoritmo 3 MAXSUM3(v, n)

1: **return** SUMPARTICION($v, 0, n - 1$)

Algoritmo 4 SUMPARTICION(v, inf, sup)

```

1: if  $inf > sup$  then
2:   return 0
3: end if
4: if  $inf = sup$  then
5:   return MAXIMO(0,  $v[inf]$ )
6: end if
7:  $m \leftarrow (inf + sup) / 2$ 
8:  $sumactual \leftarrow 0$ 
9:  $izqmax \leftarrow 0$ 
10: for  $i \in \{m, \dots, inf\}$  do
11:    $sumactual \leftarrow sumactual + v[i]$ 
12:   if  $sumactual > izqmax$  then
13:      $izqmax \leftarrow sumactual$ 
14:   end if
15: end for
16:  $sumactual \leftarrow 0$ 
17:  $dermax \leftarrow 0$ 
18: for  $i \in \{m + 1, \dots, sup\}$  do
19:    $sumactual \leftarrow sumactual + v[i]$ 
20:   if  $sumactual > dermax$  then
21:      $dermax \leftarrow sumactual$ 
22:   end if
23: end for
24:  $sumainterseccion \leftarrow izqmax + dermax$ 
25:  $maxpartizq \leftarrow$  SUMPARTICION( $v, inf, m$ )
26:  $maxpartder \leftarrow$  SUMPARTICION( $v, m + 1, sup$ )
27: return Maximo( $sumainterseccion$ , Maximo( $maxpartizq, maxpartder$ ))

```

MaxSum4: Esta es la idea más elegante pues únicamente recorremos el arreglo una vez y gracias a que descartamos el valor de la suma cuando toma un valor negativo podemos asegurar que nunca restaremos al resultado final, de modo que obtendremos al menos 0 como suma máxima. Sin embargo es importante notar que esta opción no sirve cuando todos los números del arreglo son negativos pues, como en el último de los ejemplos que ilustraban este problema más arriba, nos gustaría tener el mayor de entre esos números y no un 0.

A la izquierda se encuentra la gráfica resultante de implementar y ejecutar el algoritmo varias veces con distintas complejidades de entrada.

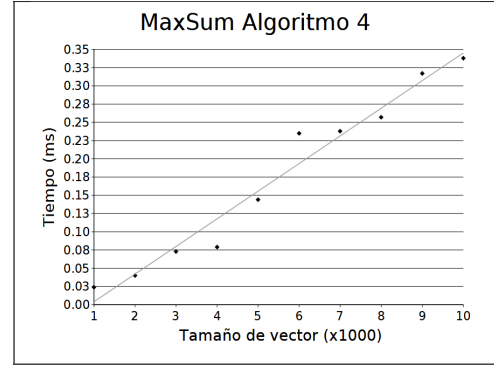


Figura 5: Gráfica de la ejecución de MaxSum4 (cada punto es el promedio de 100 ejecuciones).

Algoritmo 5 MAXSUM4(v, n)

```

1:  $sumamax \leftarrow 0$ 
2:  $sumaactual \leftarrow 0$ 
3: for  $i \in \{0, \dots, n-1\}$  do
4:    $sumactual \leftarrow sumactual + v[i]$ 
5:   if  $sumactual < 0$  then
6:      $sumactual \leftarrow 0$ 
7:   end if
8:   if  $sumactual > sumamax$  then
9:      $sumamax \leftarrow sumactual$ 
10:  end if
11: end for
12: return  $sumamax$ 

```

Entonces, tras programar y ejecutar estos algoritmos obtendremos gráficas a partir de las cuales podríamos deducir las funciones de complejidad correspondientes, a saber:

- $f(n) = n^3$ para MAXSUM1
- $g(n) = n^2$ para MAXSUM2
- $h(n) = n$ parece ser adecuada tanto para MAXSUM3 como para MAXSUM4

Lamentablemente esta forma de deducir la función de complejidad presenta varios problemas que podrían habernos llevado a conclusiones incorrectas:

Para empezar, aún si siempre utilizamos el mismo sistema operativo, el mismo lenguaje de programación, el mismo compilador y la misma computadora para ejecutarlo, nuestras mediciones podrían verse afectadas por cuestiones intrínsecas del sistema operativo como algún proceso ajeno a nuestros algoritmos que meta ruido en nuestras mediciones. Además, la cantidad de complejidades de entrada sobre las que podemos ejecutar el algoritmo siempre quedará limitada a un conjunto finito.

Para solventar estas dificultades, tomaremos otro camino: Empezaremos por definir un marco teórico que abstraiga la idea del tiempo de ejecución de su concepto físico medido en fracciones de segundo y lo vuelva dependiente únicamente del diseño de nuestro algoritmo.

Esto es muy importante, porque daremos un salto conceptual que nos permitirá teorizar sobre el comportamiento de nuestros algoritmos y compararlos entre sí.

Medición del tiempo de ejecución

Recordemos que un algoritmo está constituido por un número finito y ordenado de pasos válidos para una máquina dada y cuya ejecución termina en algún momento. Que dichos pasos sean válidos quiere decir que la máquina puede entenderlos y para ello deben estar constituidos por operaciones básicas disponibles en esa máquina. La forma de implementar el modelo de una máquina puede variar, es decir, cuando se fabrica una computadora según una especificación dada el producto físico y tangible varía según la compañía, los materiales disponibles y otros factores, pero, si de antemano sabemos que en tal modelo está especificado un conjunto de operaciones básicas podemos prescindir de la implementación física y atender únicamente a las operaciones básicas involucradas en la ejecución de un algoritmo. Así construimos un puente entre el tiempo físico medido en fracciones de segundo y un tiempo teórico medido en operaciones básicas.

En lo sucesivo consideraremos las siguientes operaciones básicas comunes a todo modelo de cómputo actual (véase [2]):

- Operación aritmética.
- Asignación a una variable.
- Llamada a una función.
- Retorno de una función.
- Comparaciones lógicas (con salto).
- Acceso a una estructura (arreglo, matriz, lista ligada, ...).

Con esto en mente, veremos tres ejemplos en los que construiremos una función $T(n)$ donde n está en los naturales y representa el tamaño de los datos de entrada, mientras que $T(n)$ representa el tiempo de ejecución para cada uno de los siguientes algoritmos medido en cantidad de operaciones básicas cuando ocurre el peor de los casos, esto es, cuando los ciclos se ejecutan sin interrupción sobre todo el rango definido:

Ejemplo 1: el ciclo **while**

Algoritmo 6 Ejemplo 1 (A)

Precondiciones: A es un arreglo de enteros

```
1:  $suma \leftarrow 0$ 
2:  $posicion \leftarrow 1$ 
3: while  $posicion \leq n$  do
4:    $suma \leftarrow suma + A[posicion]$ 
5:    $posicion \leftarrow posicion + 1$ 
6: end while
7: return  $suma$ 
```

Un ciclo **while** que se repite n veces, llevará a cabo $n + 1$ comparaciones porque antes de detenerse verifica que la condición de paro se ha cumplido; en el ejemplo anterior esta condición se cumple cuando $posicion > n$. El cuerpo del **while** se ejecuta n veces, de modo que el total de operaciones básicas realizadas en un ciclo **while** puede expresarse como:

$$\text{op_while}(n) = n(1 + \text{cuerpo}) + 1.$$

Bajo esa consideración realicemos el conteo de las operaciones básicas de nuestro algoritmo:

Cuadro 1: Conteo de operaciones básicas para el Ejemplo 1.

Línea	Número de operaciones básicas	Descripción
1	1	Asignación de variable
2	1	Asignación
3	1	Comparación
4	3	Asignación, suma y acceso al arreglo A
5	2	Asignación y suma
6	$n(1 + (2 + 3)) + 1$	Total de operaciones del while que comienza en la línea 3
7	1	Retorno

Tras sumar las operaciones básicas de las líneas 1, 2 y 7 al total de operaciones del ciclo **while** obtenemos la siguiente expresión para la función de complejidad de nuestro algoritmo:

$$T(n) = 6n + 4.$$

Ejemplo 2: el ciclo for

El siguiente algoritmo cuenta con ciclos **for** expresados en pseudocódigo. Al implementar esta estructura de control solemos escribir:

```
for (i = k; i < n; i++){
    <cuerpo del for>
}
```

De modo que en el encabezado de un bucle **for** tenemos una asignación, una comparación y un autoincremento que en realidad está compuesto por una suma y una asignación. Tanto la comparación como el autoincremento, junto a las operaciones del cuerpo, se repiten tantas veces como se repita el ciclo **for**. Por último, el procesador realiza una última comparación para verificar que se ha llegado a la condición de paro. De modo que podríamos escribir la siguiente expresión para cuantificar las operaciones de un ciclo **for** que se repite n veces:

$$\text{op_for}(n) = 1 + n(3 + \text{cuerpo}) + 1.$$

Con esto en mente obtengamos la función de complejidad del siguiente algoritmo:

Algoritmo 7 Ejemplo 2 (n)

Precondiciones: n es un entero

```
1:  $x \leftarrow 0$ 
2: for  $i \in \{1, \dots, n\}$  do
3:   for  $j \in \{1, \dots, n\}$  do
4:      $x \leftarrow x + 1$ 
5:   end for
6: end for
7: return  $x$ 
```

La expresión del número de operaciones básicas en la línea 6,

$$1 + n(3 + (5n + 2)) + 1,$$

Cuadro 2: Conteo de operaciones básicas para el ejemplo 2.

Línea	Número de operaciones básicas	Descripción
1	1	Asignación de variable
2	$1 + 3$	Las operaciones del encabezado for
3	$1 + 3$	Encabezado del bucle for
4	2	Asignación y suma
5	$1 + n(3 + 2) + 1$	Total de operaciones del for de la línea 3
6	$1 + n(3 + (5n + 2)) + 1$	Total de operaciones del for de la línea 2
7	1	Retorno

comprende todas las operaciones de los bucles **for** anidados. De modo que si a esta expresión le añadimos las operaciones básicas realizadas en las líneas 1, 2 y 7 obtenemos la siguiente función de complejidad para nuestro algoritmo:

$$T(n) = 8 + 5n + 5n^2.$$

Ejemplo 3: de nuevo el ciclo **while**

Finalmente analicemos un algoritmo en el que el ciclo **while** se repite $\log_2 n$ veces:

Algoritmo 8 Ejemplo 3 (n)

Precondiciones: $n > 0$ y n es una potencia entera de 2

```

1:  $i \leftarrow n$ 
2:  $contador \leftarrow 0$ 
3: while  $i > 1$  do
4:    $i \leftarrow i/2$ 
5:    $contador \leftarrow contador + 1$ 
6: end while
7: return  $contador$ 
```

Cuadro 3: Conteo de operaciones básicas para el ejemplo 3.

Línea	Número de operaciones básicas	Descripción
1	1	Asignación de variable
2	1	Asignación
3	1	Comparación
4	2	Asignación y división
5	2	Suma y asignación
6	$\log_2(n)(1 + 4) + 1$	Total de operaciones del while que comienza en la línea 3
7	1	Retorno

Como el total de las operaciones en el **while** es

$$1 + 5\log_2(n),$$

a ese valor añadimos las operaciones de las líneas 7, 2 y 1 para obtener la siguiente función de complejidad:

$$T(n) = 4 + 5 \log_2(n).$$

Es fundamental expresar que **las funciones que hemos obtenido dependen de las operaciones básicas que elegimos**. Si añadimos o quitamos operaciones nuestras funciones cambiarán. Sin embargo **el grado del polinomio que las define no disminuirá mientras no descartemos la operación esencial**, es decir, aquella operación de la que depende el proceso entero del algoritmo. Para ejemplificar esto regresemos al ejemplo 1:

Algoritmo 9 Ejemplo 1 (A), considerando la operación esencial

Precondiciones: A es un arreglo de enteros

```

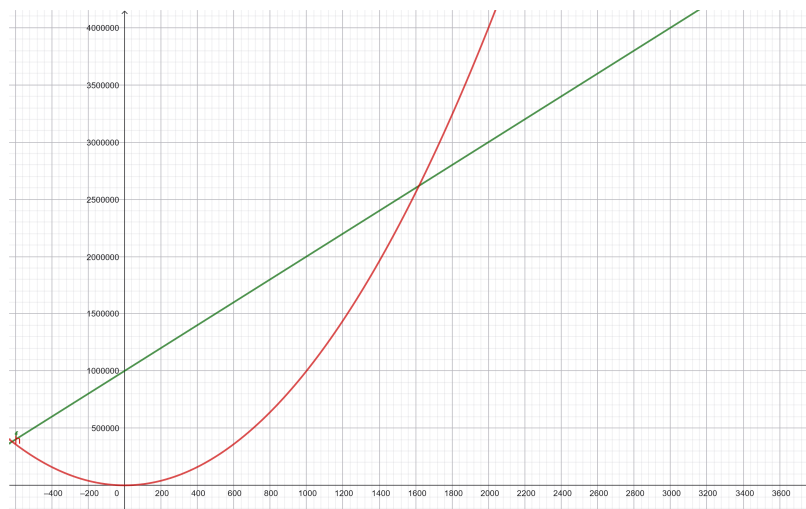
1:  $suma \leftarrow 0$ 
2:  $posicion \leftarrow 1$ 
3: while  $posicion \leq n$  do
4:    $suma \leftarrow suma + A[posicion]$ 
5:    $posicion \leftarrow posicion + 1$ 
6: end while
7: return  $suma$ 

```

Si consideramos que la operación esencial es la suma, nuestro conteo se reduce a las dos sumas del bucle **while** que se repiten n veces, es decir, nuestra función de complejidad es:

$$T(n) = 2n.$$

El hecho de que el grado del polinomio se mantenga sugiere algo sobre cómo podemos comparar distintos algoritmos según sus funciones de complejidad. En la próxima nota abordaremos este tema. Por ahora exploremos en la siguiente gráfica como las funciones cuadráticas crecen más que las funciones lineales. Por más grande que sea la constante que le sumemos o multipliquemos, una función lineal siempre se verá superada por una cuadrática. A continuación vemos como una función lineal multiplicada por mil y a la que le hemos sumado una constante de un millón $f(x) = 1000x + 1000000$ se ve superada por una función cuadrática $h(x) = x^2$ en algún punto cercano a $x = 1618$. De este modo un algoritmo que tenga una función de complejidad que consista en un polinomio de grado 1, siempre será mejor opción al trabajar con complejidades de entrada suficientemente grandes que otro algoritmo cuya función de complejidad esté constituida por un polinomio cuadrático.



Referencias

- [1] José Galaviz Casas. *Estructuras de datos y análisis de algoritmos: Una introducción usando Java*. Facultad de Ciencias, UNAM, Ciudad de México, México, 2012.
- [2] María del Carmen Gómez Fuentes and Jorge Cervantes Ojeda. *Introducción al Análisis y al Diseño de Algoritmos*. Universidad Autónoma Metropolitana, jun 2014.